

TEAM MOIRAI

ALLOC

IT 기업을 위한 역량·일정 기반
프로젝트 관리 플랫폼





1	프로젝트 개요	01
2	프로젝트 핵심 기능	06
3	기술 설계 및 구현	18
4	프로젝트 수행 절차 및 방법	29
5	프로젝트 결과 및 회고	31

ALLOC 프로젝트의 기획 의도와 배경을 설명합니다.

문제 인식부터 프로젝트 목표까지,
전체 방향성을 이해할 수 있도록 구성했습니다.

CHAPTER 1

OVERVIEW

프로젝트 개요

TEAM INTRODUCTION

팀원 소개

이름	역할	담당업무
김현수	팀장, 백엔드, ML	RAG 기반 리스크 관리 기능, SW 아키텍처, 간트 차트 기능, 주간 보고, 회의록 기능
김명진	백엔드, 보안	인증 및 인가 기능 , 관리자 기능, 공통 더미 데이터 생성
김동리	백엔드, ML	프로젝트 스쿼드 추천 기능, 프로젝트 등록 기능
박연수	기획, 프론트엔드	VUE.JS 기반 프론트엔드 구축, 화면 설계 및 UI/UX 구현, PPT 제작
배창민	백엔드, 인프라	SQS 기반 알림 기능, CI/CD 파이프라인 구축, 프로젝트 캘린더 기능, AWS 서버 관리 및 장애 대응
정희영	멘토	기술 멘토링 및 시스템 아키텍처 리뷰

OVERVIEW

ALLOC 프로젝트 개요



ALLOC은 C언어에서 MALLOC으로 메모리를 할당하듯,
프로젝트의 요구사항과 일정에 맞춰 인력을 합리적으로 배치하고
프로젝트 일정과 리스크 관리를 보조하는 프로젝트 관리 플랫폼입니다.

BACKGROUND

주제 선정 배경

현업에서 반복되는 PM의 고민

"프로젝트 성패를 결정짓는 인력 배치와 일정 관리, 왜 여전히 데이터가 아닌 직관과 경험에만 의존해야 할까요?"



김PM

"객관적 데이터 기반의 인력 매칭 기준 부재"

프로젝트별 상이한 기술 스택과 개인별 역량(기술·경험·가용성)을 한눈에 비교할 지표가 없어 최적의 인재를 놓치고 있습니다.



OIPM

"특정 인력 의존도 심화 및 리소스 불균형"

검증된 특정 인원에게 업무가 집중되는 병목 현상은 개인의 번아웃을 넘어 조직 전체의 생산성 저하와 일정 지연으로 이어집니다.



박PM

"암묵적인 리스크 관리로 인한 사후 수습 반복과 뒤늦은 후회"

애자일/스크럼에서는 위험 관리가 암묵적으로 행해지고 있어 추적이 힘든 상황입니다.

PROJECT OBJECTIVE

기획의도



기술 요구사항-인력 역량 불일치

프로젝트에 필요한 기술 스택과 실제 투입 가능한 인력의 역량 간 정합성을 정밀하게 판단하기 어려워, 기술 수준·경험·유사 프로젝트 이력이 체계적으로 관리되지 않는 문제가 발생



인력 배치의 비효율과 편중 구조

검증된 일부 인력에게 업무가 반복적으로 집중되면서 과부하와 병목 현상이 발생하고, 조직 전체의 인력 활용 효율이 저하



사전 중심 인력 배치의 한계

프로젝트 초기 단계에서의 인력 배치는 일정 지연과 같은 일부 리스크를 예방할 수 있으나, 진행 이후 발생하는 변수를 충분히 대응하지 못함



지속적인 프로젝트 리스크 관리 필요

프로젝트 진행 중 발생하는 일정 변경, 규정 위반, 비용 증가 등의 리스크에 대응하기 위해 인력 배치 이후에도 지속적인 리스크 관리 체계가 필요

COMPETITIVE DIFFERENTIATION

기존 서비스와의 차별성



JIRA

개발·IT 팀 중심 애자일 프로젝트 관리 도구

- 이슈와 스프린트 기반의 개발 프로세스 관리에 특화
- 개발 진행 상황 추적 및 시각화에 강점
- 글로벌 시장에서 널리 사용되는 표준 도구



FLOW

국내 시장 중심 올인원 협업 툴

- 프로젝트·업무 관리, 파일 공유, 커뮤니케이션 기능 통합
- 전사 협업과 업무 가시성 확보에 초점
- 비개발 조직까지 폭넓게 활용 가능



ALLOC

ALLOC은 프로젝트 단위의 요구사항을 출발점으로 설계된 PM 중심 관리 도구

프로젝트 일정과 팀원의 기술 스택, 가용 일정을 종합적으로 고려하여
인력 배치 의사결정을 보조하고,
프로젝트 수행 과정에서 축적되는 일정·리스크·변경 이력을 기반으로
주간 단위 운영 요약과 일정 지연 요소를 구조적으로 관리할 수 있도록 지원

CHAPTER 2

CORE FEATURES

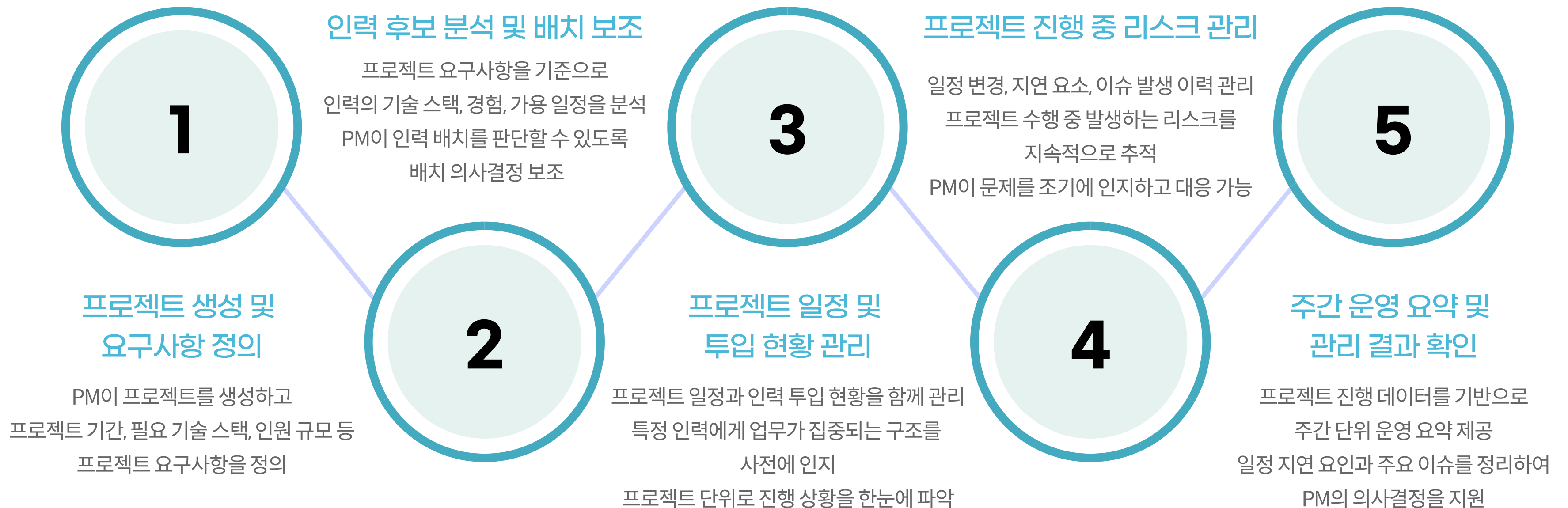
프로젝트 핵심 기능

ALLOC 프로젝트의 핵심 기능을 소개합니다.

프로젝트 단위의 인력 배치, 일정 관리, 리스크 대응 기능을 중심으로
PM의 의사결정을 지원하는 주요 기능을 설명합니다.

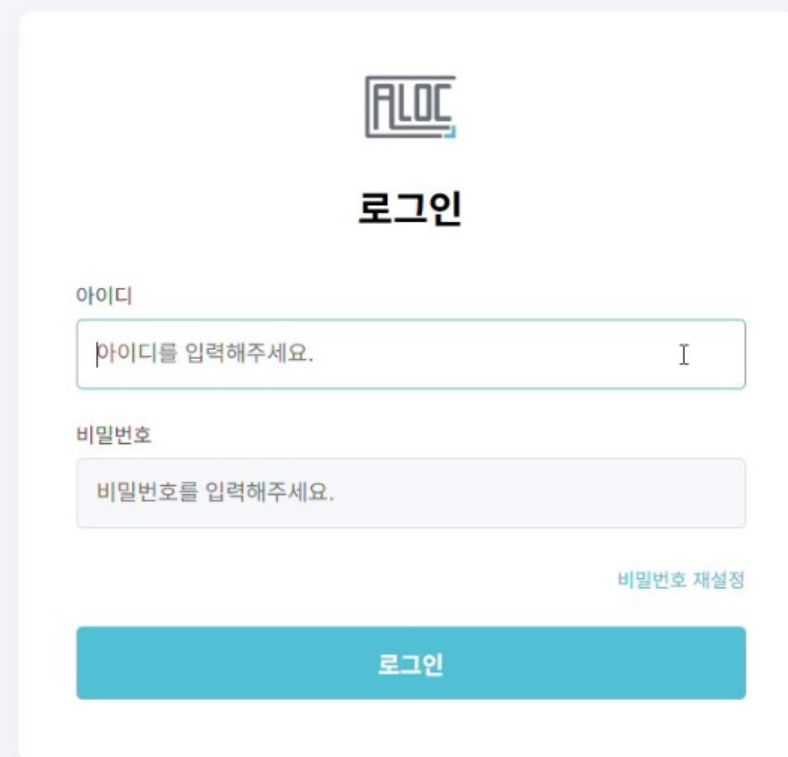
PROJECT FLOW

프로젝트 흐름



LOGIN & PASSWORD RESET

로그인 및 비밀번호 재설정



The image shows a login and password reset form for the ALLOC system. At the top, there is the ALLOC logo and the word '로그인' (Login). Below this, there are two input fields: one for '아이디' (ID) with the placeholder text '아이디를 입력해주세요.' (Please enter your ID), and another for '비밀번호' (Password) with the placeholder text '비밀번호를 입력해주세요.' (Please enter your password). To the right of the password field, there is a link for '비밀번호 재설정' (Reset Password). At the bottom of the form, there is a large blue button labeled '로그인' (Login).

편의성과 보안을 고려한 인증 구조

- JWT 기반 인증·인가 구조 적용
- REFRESH TOKEN을 REDIS에 저장하여 토큰 관리 및 보안 강화

비밀번호 재설정 기능

- 등록된 이메일 인증 기반 비밀번호 재설정 제공
- 계정 접근이 어려운 상황에서도 사용자가 직접 안전하게 비밀번호 변경 가능

보안 설계

- HTTP-ONLY COOKIE를 활용한 토큰 저장
- XSS 공격으로 인한 토큰 탈취 위험 최소화

CREATE PROJECT

프로젝트 등록하기

ALLOC

홈

프로젝트

인력

문서

프로젝트 관리

프로젝트 목록

+ 프로젝트 등록

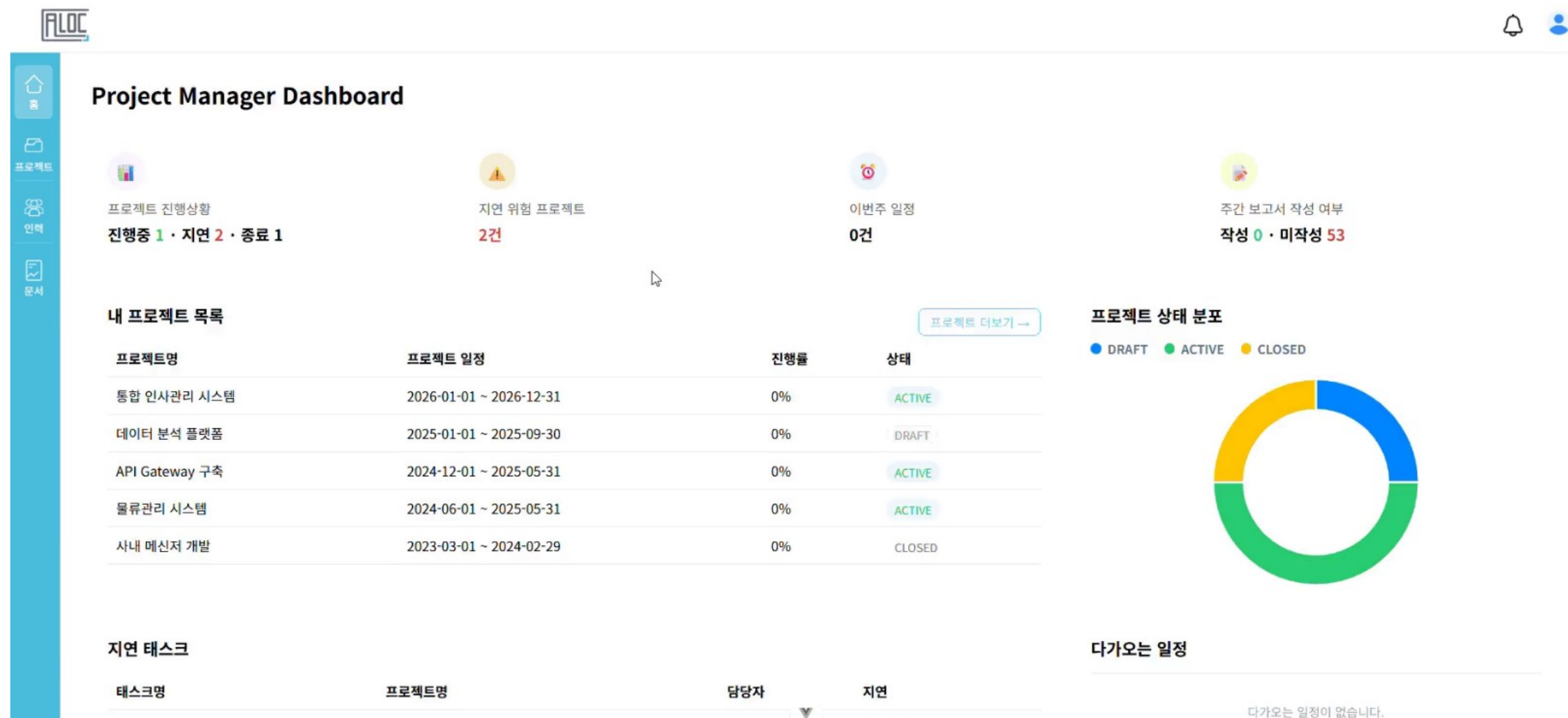
프로젝트 이름	프로젝트 기간	상태	진행률	문서 상태	리스크
통합 인사관리 시스템	2026-01-01 ~ 2026-12-31	ACTIVE	0%	-	HIGH
데이터 분석 플랫폼	2025-01-01 ~ 2025-09-30	DRAFT	0%	-	HIGH
API Gateway 구축	2024-12-01 ~ 2025-05-31	ACTIVE	0%	-	HIGH
물류관리 시스템	2024-06-01 ~ 2025-05-31	ACTIVE	0%	-	HIGH
사내 메신저 개발	2023-03-01 ~ 2024-02-29	CLOSED	0%	-	HIGH
인재추천 테스트	2026-01-01 ~ 2026-01-31	DRAFT	0%	-	HIGH
테스트	2026-01-01 ~ 2026-01-31	DRAFT	0%	-	HIGH
test	2026-01-01 ~ 2026-01-31	DRAFT	0%	-	HIGH
test2	2026-01-01 ~ 2026-12-31	DRAFT	0%	-	HIGH
test3	2026-02-02 ~ 2026-02-28	DRAFT	0%	-	HIGH
test123	2026-02-01 ~ 2026-02-28	DRAFT	0%	-	HIGH

구조화된 프로젝트 등록 흐름

- 프로젝트 기본 정보
(프로젝트명, 기간, 유형, 설명)를 시작으로
필요 직군과 세부 기술 스택을 단계적으로 입력
- 상단에서 하단으로 이어지는 입력 흐름을 통해
복잡한 프로젝트 정보를 구조화된 데이터로 수집
- 설계 의도
 - 사용자의 입력 부담을 줄이면서도
인재 추천 및 프로젝트 관리에 필요한
핵심 정보를 누락 없이 확보
 - 이후 인력 배치, 일정 관리, 리스크 분석 등
후속 기능과 자연스럽게 연결되는 등록 구조로
설계

PROJECT LIST & DETAILS

프로젝트 목록 및 상세보기



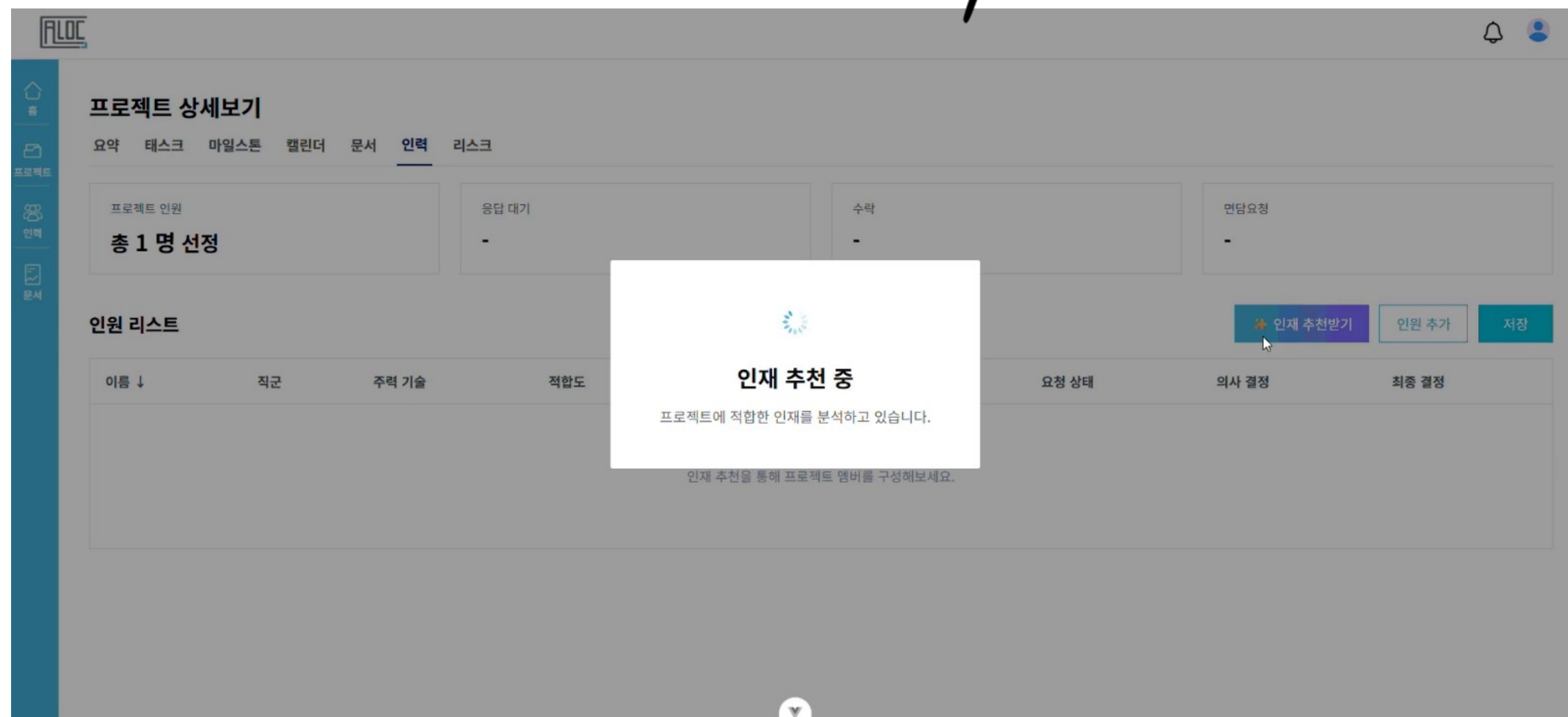
주요 업무 흐름 통합

- 프로젝트 상세 화면을 기준으로 문서 작성, 리스크 리포트 생성, 일정 관리 등 주요 업무를 즉시 수행 가능
- 화면 이동이나 컨텍스트 전환 없이 업무 흐름이 자연스럽게 이어지도록 UI를 설계
- 설계 의도
 - 프로젝트 정보를 분산된 메뉴가 아닌 하나의 컨텍스트에서 통합 관리
 - PM뿐만 아니라 팀원 모두가 현재 프로젝트 상황을 빠르게 이해하고 협업할 수 있도록 설계

RESOURCE RECOMMENDATION

인력(인재 추천) - 1

- PM이 슬라이더를 통해 가중치 조절 기능
- 기술 적합도, 경험 적합도, 투입 가능성
- 현업 판단을 직접 반영한 최종 후보 선정 가능



인력 적합도 평가 요소 (3가지)

1. SKILL FIT (기술 적합도)

- 프로젝트 요구 기술을 실제로 수행할 수 있는지 평가
- 요구 숙련도 대비 실제 숙련도 비율 기반 점수 산정
- 기술 역량 MISMATCH는 프로젝트 실패의 주요 원인

2. EXPERIENCE FIT (경험 적합도)

- 단순 경력 연차가 아닌 유사 프로젝트 경험 중심 평가
프로젝트 유형 경험, 최근 수행 이력, 프로젝트 규모
경험 반영

3. AVAILABILITY FIT (투입 가능성)

- 일정상 실제 투입 가능한지 판단
- 동시 진행 프로젝트 수 기반 점수 산정
- 3개 이상: 배제 / 2개: 감점 / 0~1개: 최대 점수

RESOURCE RECOMMENDATION

인력(인재 추천) - 2

프로젝트 상세보기

요약 태스크 마일스톤 캘린더 문서 **인력** 리스크

프로젝트 인원: 총 3 명 선정

응답 대기: 2

수락: 1

면담요청: 0

인원 리스트

이름 ↓	직군	주력 기술	적합도	단가	현재 투입 상태	요청 상태	의사 결정	최종 결정
이프로	Backend	Docker	0%	6,500,000	투입중	수락	-	투입
김백엔	Backend	Java	44%	5,500,000	대기중	투입 요청됨	면담 요청 수락	-
이리엑트	Frontend	HTML/CSS	11%	3,500,000	대기중	투입 요청됨	-	-

인력 추천 프로세스

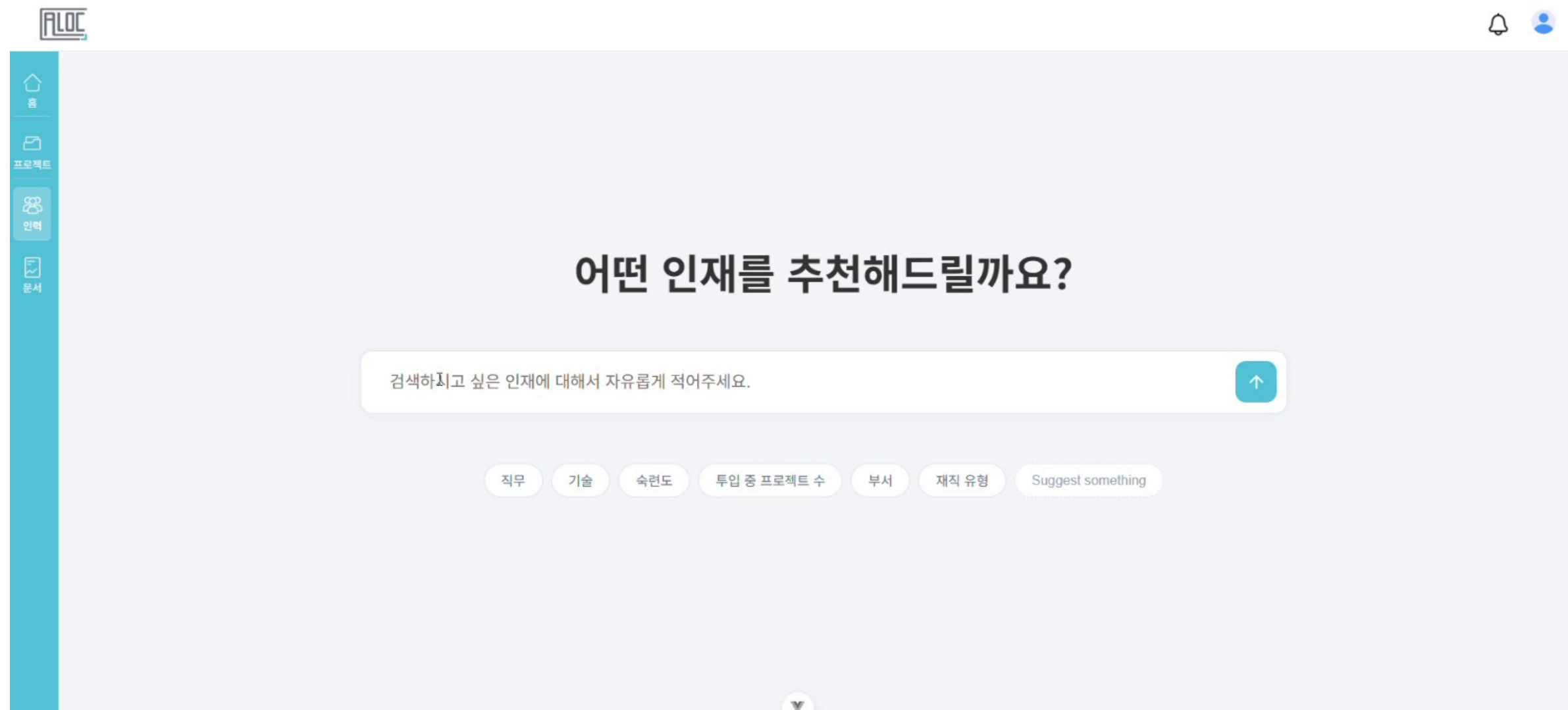
1. 직군별 인력 POOL 조회
2. SKILL / EXPERIENCE / AVAILABILITY 점수 계산
3. 프로젝트 유형별 기본 가중치 적용
4. 직군별 3배수 후보 리스트 제공

IMPACT

- 인력 선정을 감이 아닌 데이터 기반 의사결정으로 전환
- 완전 자동 추천이 아닌 의사결정 보조 구조
- 가중치 기반 필터링으로 시작하여, 향후 AI 확장 가능한 구조로 설계

TALENT SEARCH (NLP)

인재 검색(자연어 기반)

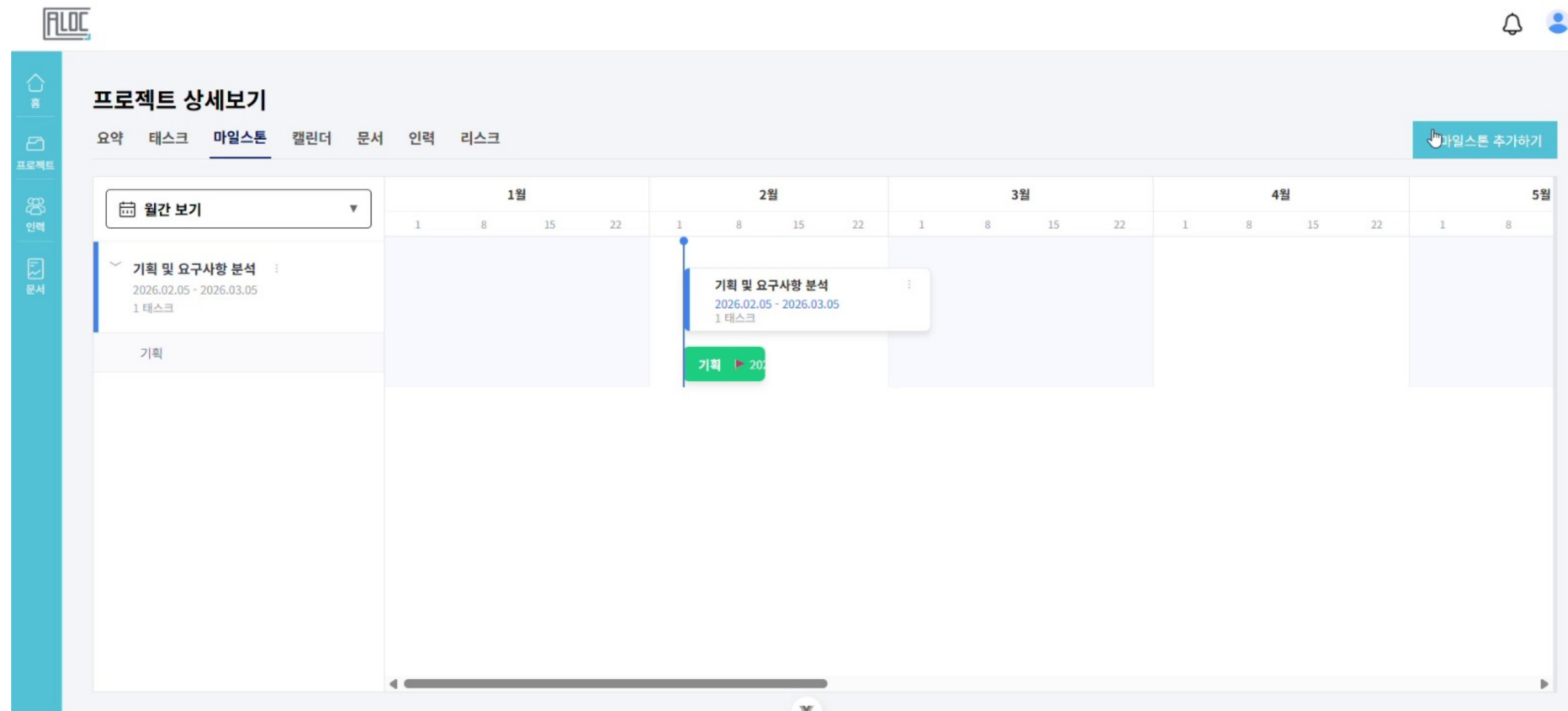


OPENSEARCH 기반 인재 검색 기능

- 기술 선택
 - 인재 검색 기능 구현을 위해 OPENSEARCH 도입
 - 단순 키워드 검색이 아닌, 조건 기반 필터링 + 자연어 기반 유사도 검색을 함께 적용
- 검색 방식
 - 조건 기반 필터링
 - : 직무, 기술, 숙련도, 투입 중 프로젝트 수 등 명확한 조건 검색
 - 자연어 기반 검색
 - : “백엔드 경험 있는 JAVA 개발자”와 같은 자연어 입력을 통한 유사 인재 검색 지원

TASKS & MILESTONES

간트 차트

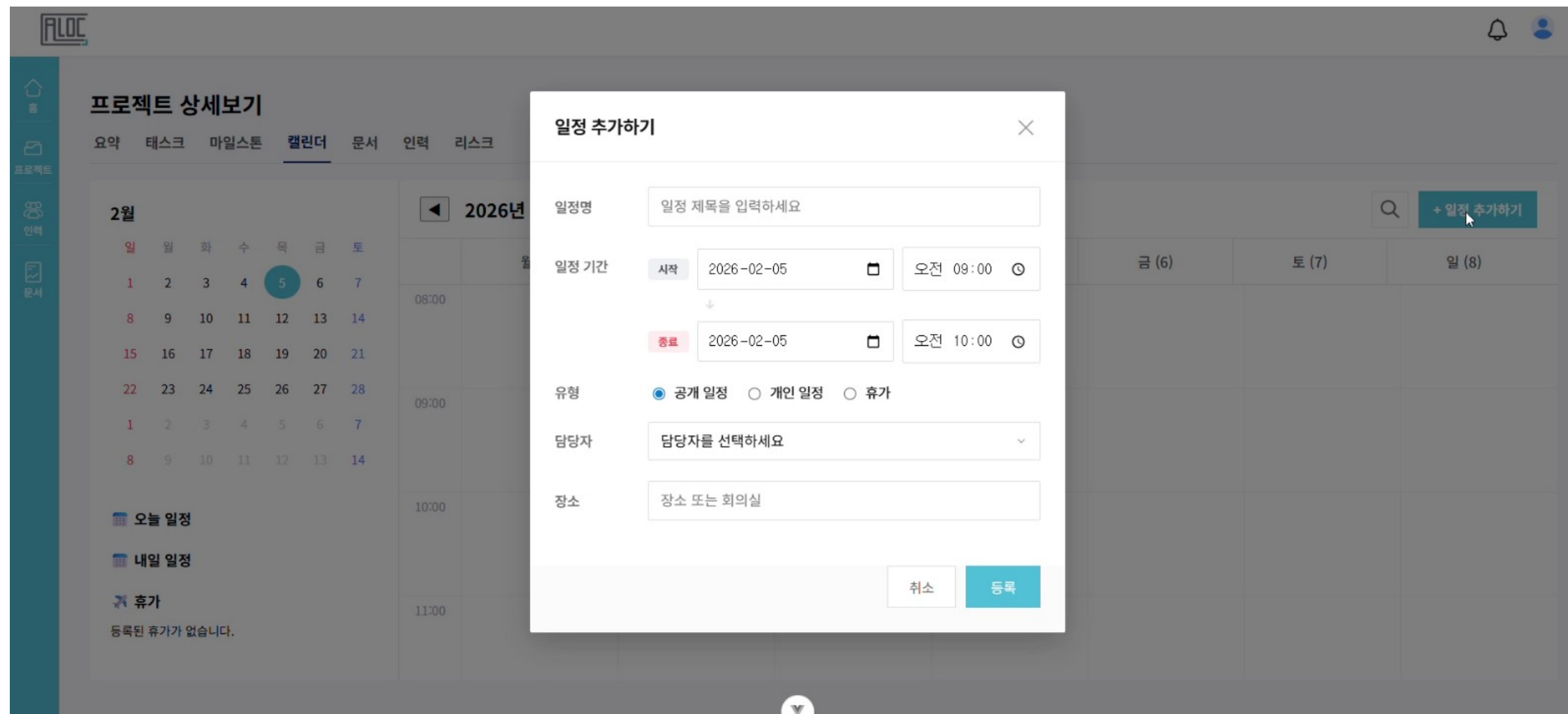


애자일한 마일스톤 설정, 태스크 분배

- 마일스톤 설정
 - 프로젝트 진행 단계를 마일스톤 단위로 구조화하여 관리
 - 모달 기반 마일스톤 등록 UI를 제공해 화면 이동 없이 빠른 등록 가능
 - 스프린트 단위(기간 중심)로 마일스톤을 설정하여 애자일 개발 흐름에 맞는 일정 관리 지원
- 태스크 등록 및 분배
 - 마일스톤 하위에 태스크를 등록하여 작업 단위 세분화
 - 태스크별 담당자, 기간, 상태를 설정하여 역할과 책임 명확화
 - 태스크 진행 상태를 기반으로 프로젝트 전체 진행 상황을 한눈에 파악 가능

CALENDAR

캘린더



프로젝트 캘린더 기반 일정 관리

- 일정 등록 방식
 - 캘린더 화면에서 모달 형태로 일정 등록 가능
 - 화면 이동 없이 빠르게 일정 추가·수정 가능
 - 일정명, 기간, 유형, 장소 등 필수 정보 중심 입력
- 일정 유형 분리 관리
 - 개인 일정: 개인 업무 및 편의를 위한 일정으로, 다른 팀원에게 공유되지 않는 개인 전용 일정
 - 공유 일정: PM이 등록하는 프로젝트 일정으로, 프로젝트 팀원 전체에게 영향을 주는 일정
- 개인 일정과 프로젝트 공유 일정을 명확히 분리하여 협업 일정 혼선 방지
- 단순 일정 기록이 아닌, 인력 가용성 판단 및 리소스 관리에 활용 가능한 일정 데이터 확보

DOCUMENTS (WEEKLY REPORTS & MEETING MINUTES)

문서 (주간보고 & 회의록)

🏠

📁 프로젝트

👥 인력

📄 문서

프로젝트 상세보기

요약

태스크

마일스톤

캘린더

문서

인력

리스크

주간보고 목록

회의록 목록

주간보고 ID

생성 날짜

최신 수정 날짜

상태

2026년 2월 1주차	20	2026. 02. 04	-	DRAFT
2026년 1월 5주차	18	2026. 01. 30	-	DRAFT
2026년 1월 5주차	17	2026. 01. 30	-	DRAFT
2026년 1월 5주차	12	2026. 01. 29	2026. 01. 30	DRAFT
2026년 1월 5주차	11	2026. 01. 29	-	DRAFT
2026년 1월 5주차	9	2026. 01. 29	-	DRAFT
2026년 1월 5주차	8	2026. 01. 29	2026. 01. 29	DRAFT
2026년 1월 5주차	7	2026. 01. 29	2026. 01. 29	DRAFT

🔔

👤

주간보고/회의록 생성

프로젝트 문서 관리 (주간보고 · 회의록)

- 주간보고 관리
 - 프로젝트 주차 기준 주간보고 자동 생성
 - 생성일, 수정일, 상태(DRAFT) 기반 진행 이력 관리
 - 임시 저장(DRAFT) 지원으로 자유로운 작성·수정
- 회의록 관리
 - 프로젝트 회의를 회의록 단위로 기록 및 관리
 - 주요 논의 사항과 결정 내용을 문서로 정리
- 보고·회의 내용을 팀 단위 공식 문서로 관리
- 프로젝트 히스토리와 의사결정 흐름을 추적할 수 있도록 설계

RISK

리스크 - PMO 위험 관리 지침서 업로드

리스크 관리 (PMO 위험 관리 지침서)

- 리스크 관리 지침서 업로드
 - 관리자 계정을 통해 PMO가 작성한 프로젝트 위험 관리 지침서 업로드
 - 업로드 즉시 프로젝트에 위험 관리 기준으로 활용 가능
 - 업로드된 지침서는 목록 형태로 관리 및 상태 확인 가능
- 설계 의도
 - 프로젝트별 리스크 관리 기준을 문서로 일원화하여 관리
 - PM 개인 경험에 의존하지 않고 조직 차원의 리스크 관리 기준을 프로젝트에 적용할 수 있도록 설계

RISK

리스크 - 리스크 리포트 생성 및 모니터링

리스크 관리

- RAG 기반 리스크 리포트 생성
 - 정성적 리스크 분석 방식 채택
 - 5×5 PI 매트릭스 활용
 - JIRA에서 사용하는 정량적 리스크 분석 기법을 모델 기반으로 시뮬레이션하여 리스크 도출
 - 충분한 과거 리스크 데이터가 없어도 분석 가능
- RAG 활용 리스크 분석 고도화
 - RAG를 활용하여 리스크 분석 품질 향상
 - PMO가 작성한 CMMI, ISO 등 국제 표준 기반 위험 관리 지침서를 바탕으로 리스크 징후를 식별하고 PI 점수 계산

CHAPTER 3

TECHNICAL DESIGN & ARCHITECTURE

기술 설계 및 구현

ALLOC 프로젝트의 기술 설계 전반을 소개합니다.

프로젝트에서 실제로 사용한 기술 스택을 중심으로,
프론트엔드, 백엔드, 데이터베이스, 인프라 구성과
각 기술이 구현 과정에서 어떻게 활용되었는지를 설명합니다.

SW ARCHITECTURE

SW 아키텍처

FRONTEND

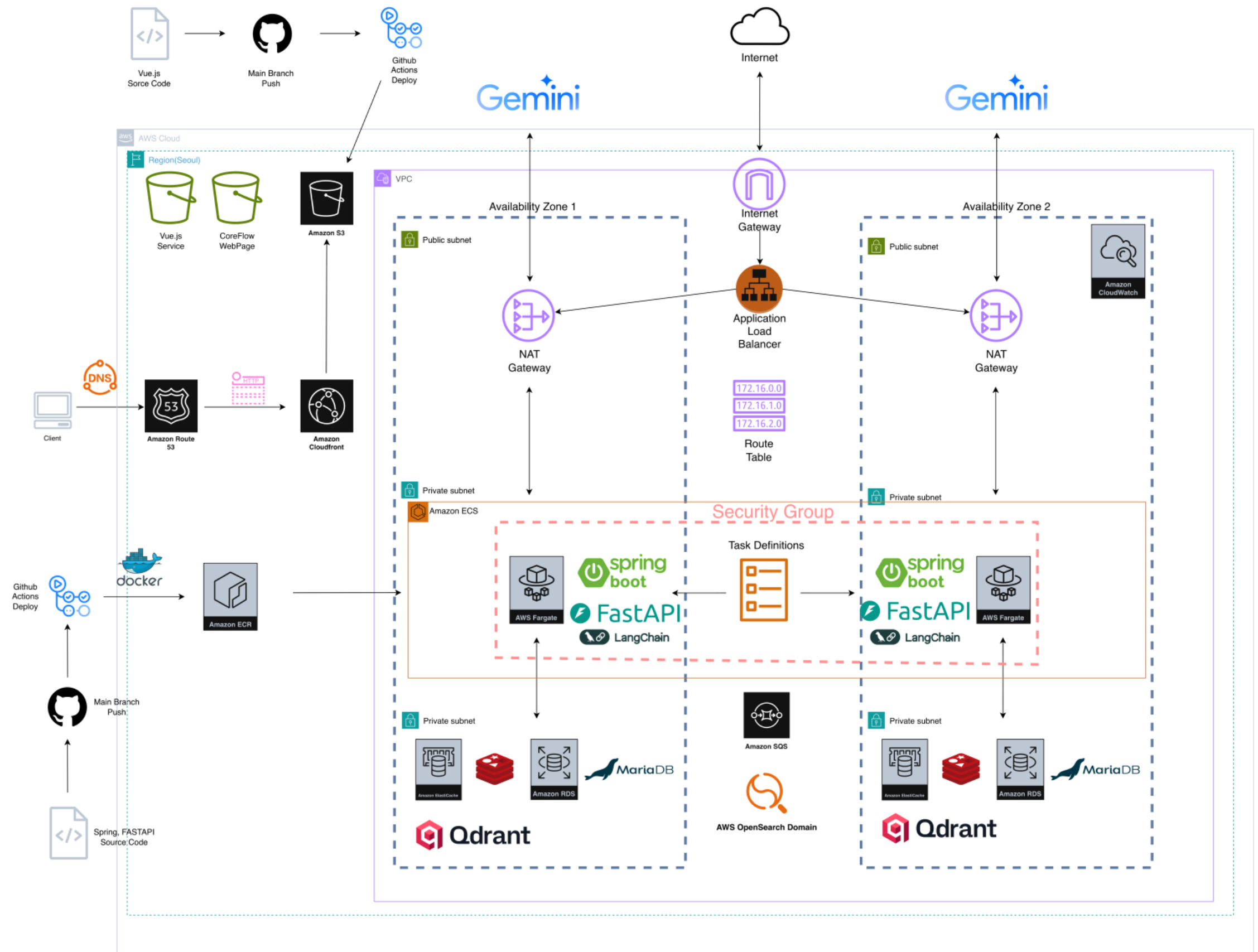
- CLIENT가 브라우저로 접속
- ROUTE 53에서 도메인 요청을 CLOUDFRONT로 라우팅
- CLOUDFRONT가 정적 화면은 캐시/S3에서 응답하고, API 요청은 백엔드로 전달

BACKEND

- 멀티 AZ 분산을 통해 장애 격리, 고가용성 지향
- SPRING BOOT에서는 인증/인가, 프로젝트 등록 및 관리, 일정관리 기능 제공
- FASTAPI에서 RAG 리스크 리포트 서비스, PDF 추출 서비스 제공

DATABASE

- ELASTIC CACHE에서 REDIS CACHING 사용
- RDBMS로는 AWS RDS, VECTOR STORE는 QDRANT, 검색 엔진으로는 OPENSEARCH 사용



ERD MODELING

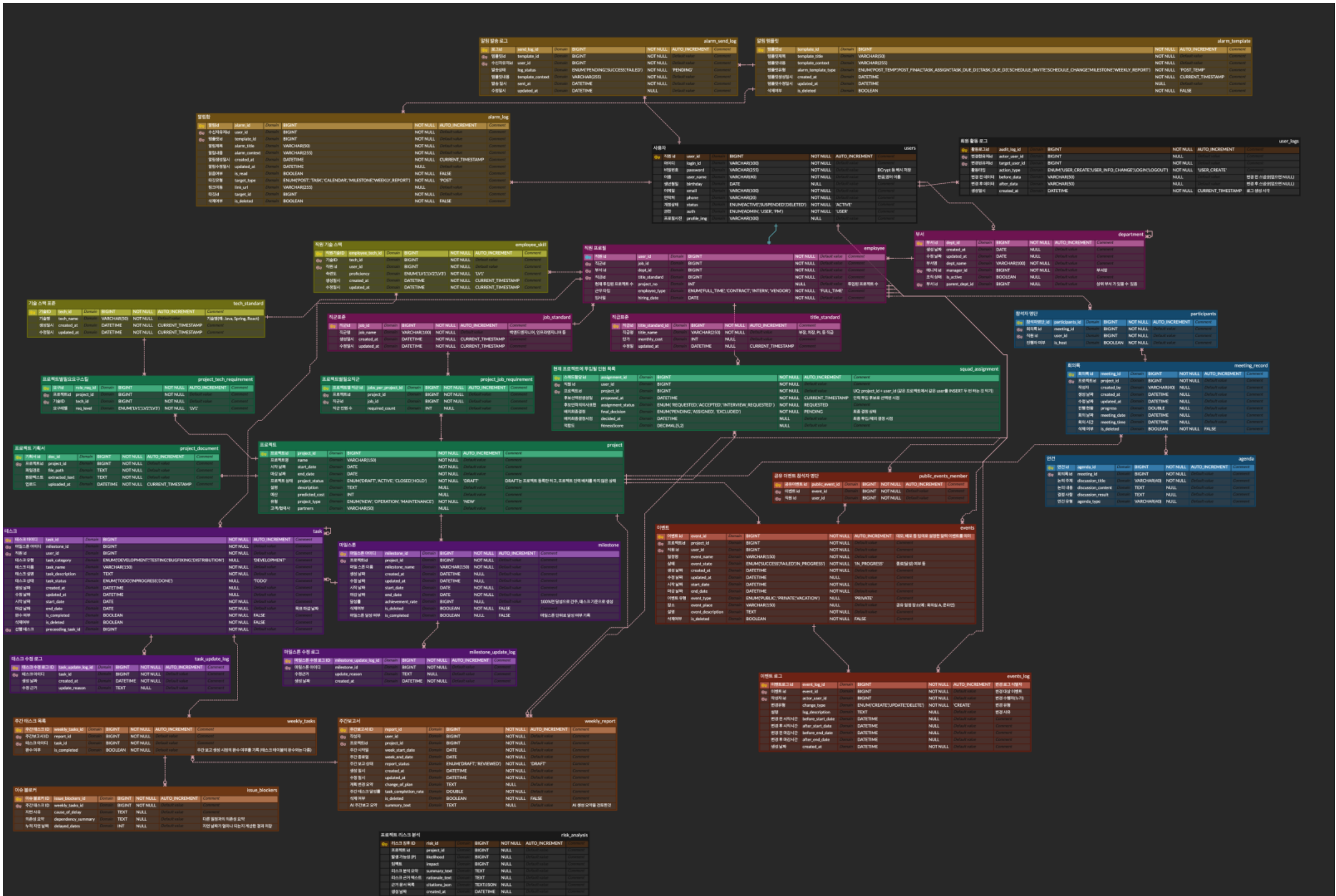
ERD 모델

프로젝트 관리 중심 ERD 설계

- 프로젝트 진행 과정에서 발생하는 업무·일정·인력·리스크·의사결정 이력을 하나의 시스템에서 통합 관리하기 위한 ERD 설계
- 단순 CRUD가 아닌 진행 상태 추적, 변경 이력 관리, 로그 기반 설계에 초점

도메인 분리 구조

- 프로젝트를 중심으로
업무(TASK/MILESTONE), 일정(EVENT),
인력(EMPLOYEE), 보고(WEEKLY REPORT),
알림/로그 도메인을 분리
- 각 도메인은 관계를 유지하되 역할과 책임을
명확히 분리



TECHNOLOGY STACK

사용 기술 스택

Design



Database



Frontend



Collaboration



Backend



CI/CD



AI



FRONTEND TECHNOLOGY

프론트엔드 설계 및 구현

Frontend Directory Structure

```

text

src
├─ api/           # API 요청 관련 파일
├─ assets/        # 이미지 및 정적 리소스
├─ components/    # 공통 UI 컴포넌트
│   └─ common/
│       └─ icons/
├─ layouts/       # 레이아웃 컴포넌트
│   └─ AdminLayout.vue
│   └─ DefaultLayout.vue
├─ lib/           # 공통 라이브러리 설정
│   └─ axios.js
├─ pages/         # 페이지 단위 컴포넌트
├─ router/        # 라우터 설정
├─ stores/        # 전역 상태 관리 (Pinia)
│   └─ auth.js
│   └─ notification.js
│   └─ pinia.js
├─ App.vue        # 루트 컴포넌트
└─ main.js        # 애플리케이션 엔트리 포인트
    
```



BACKEND TECHNOLOGY

백엔드 설계 및 구현

Backend Directory Structure (Domain-based + CQRS)

```

text

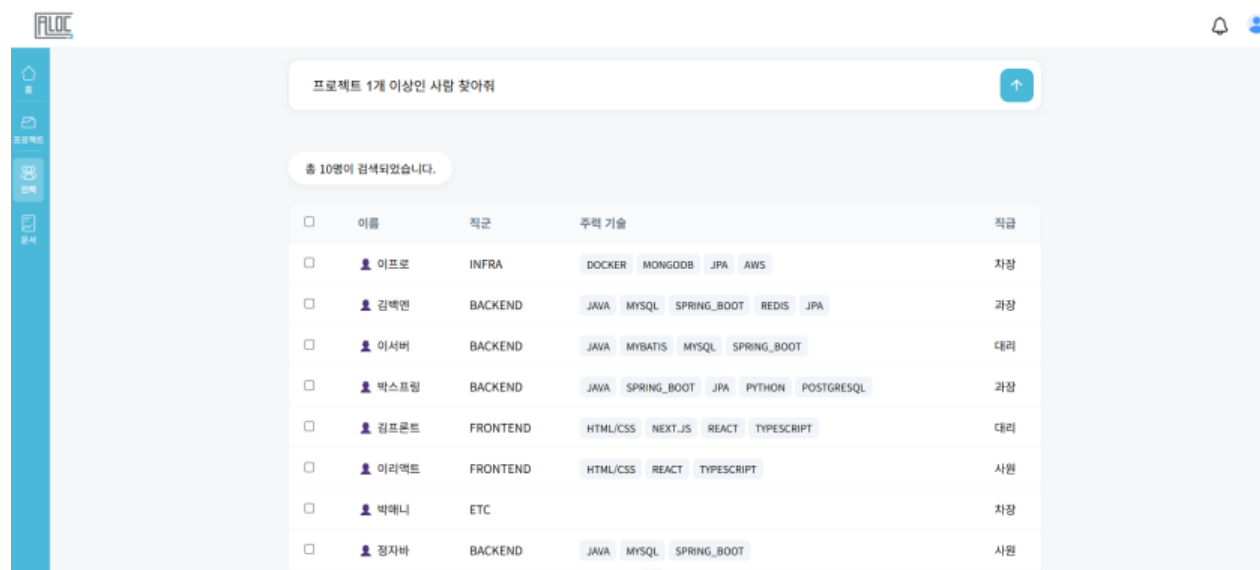
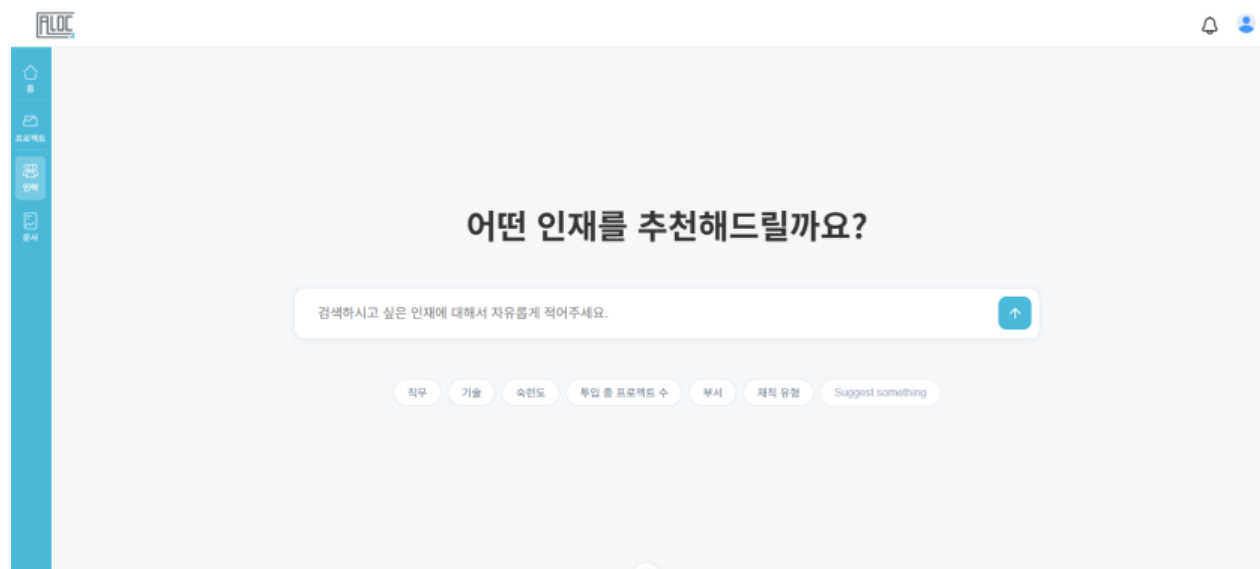
moirai
├─ alloc
│   ├── admin/      # 관리자 기능 도메인
│   ├── auth/       # 인증 / 인가 도메인
│   ├── calendar/   # 일정 관리 도메인
│   ├── common/     # 공통 설정 및 유틸
│   ├── gantt/      # 간트 차트 도메인
│   ├── home/       # 대시보드 / 홈 화면 도메인
│   ├── hr/         # 인사 관리 도메인
│   ├── management/ # 인력 배치 / 관리 도메인
│   ├── meeting/    # 회의록 도메인
│   ├── notification/ # 알림 도메인
│   ├── profile/    # 사용자 프로필 도메인
│   ├── project/    # 프로젝트 도메인
│   ├── report/     # 보고서 도메인
│   └─ search/      # 인재 검색 도메인
│       ├── command/ # 검색 인덱싱 및 데이터 변경 (CQRS - Command)
│       └─ query/    # 검색 조회 및 검색 API (CQRS - Query)

```



OPENSEARCH TALENT SEARCH

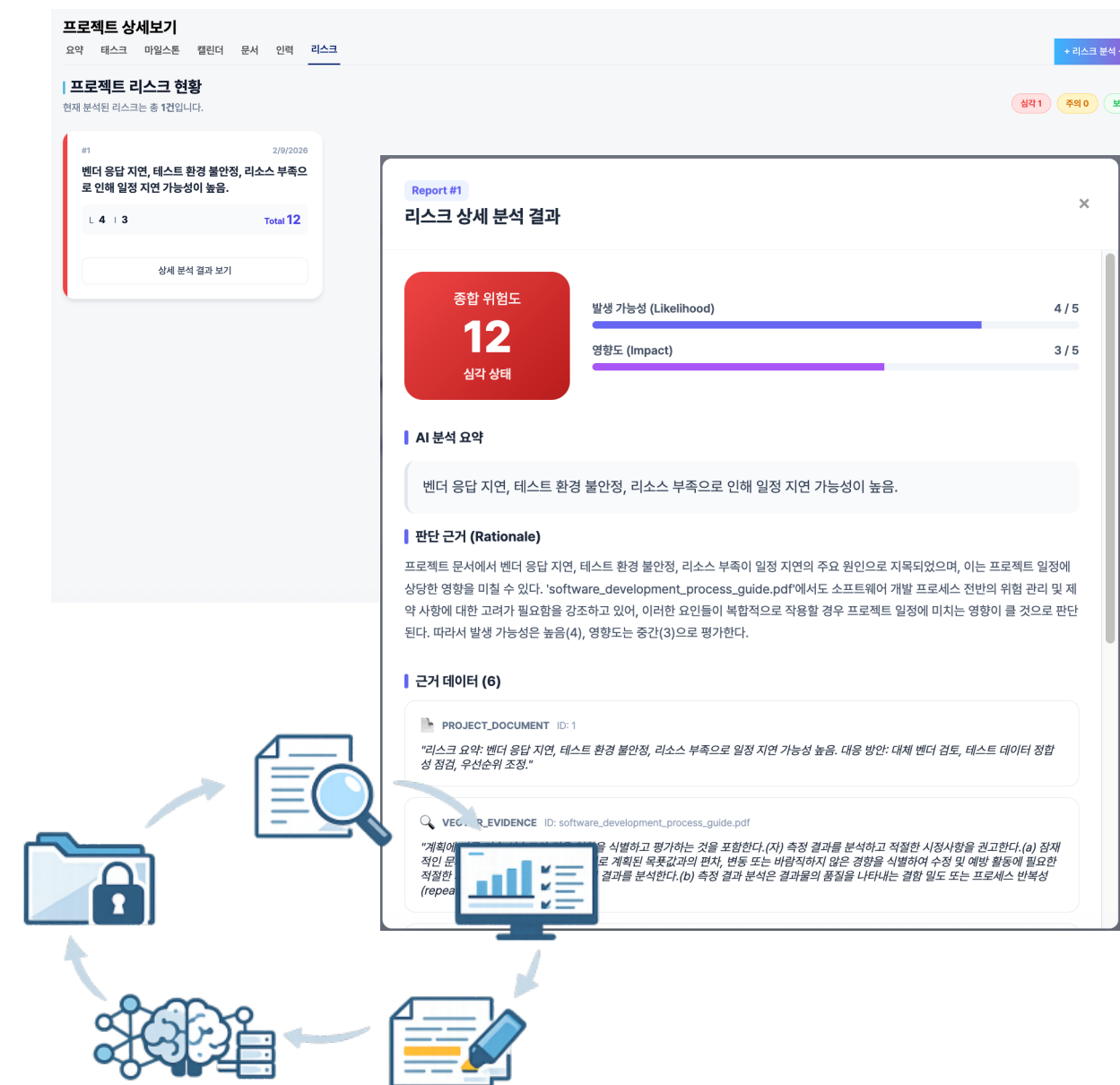
OPENSEARCH 인재 검색



ALLOC에서는 인재 검색 기능 구현을 위해 OPENSEARCH를 도입
검색 대상 데이터는 사용자 기본 정보, 기술 스택, 프로필 요약 등으로 구성되며,
조건 기반 필터링과 자연어 기반 유사도 검색을 함께 적용

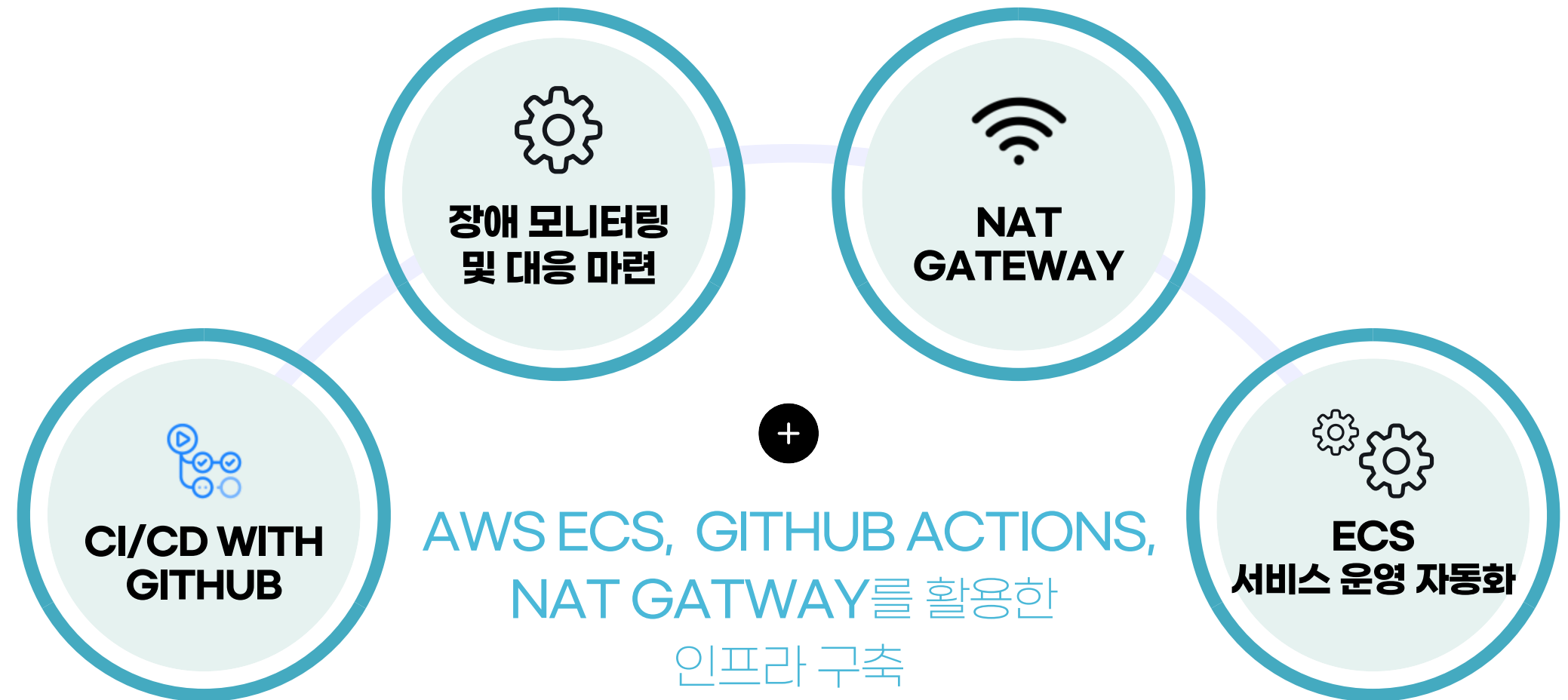
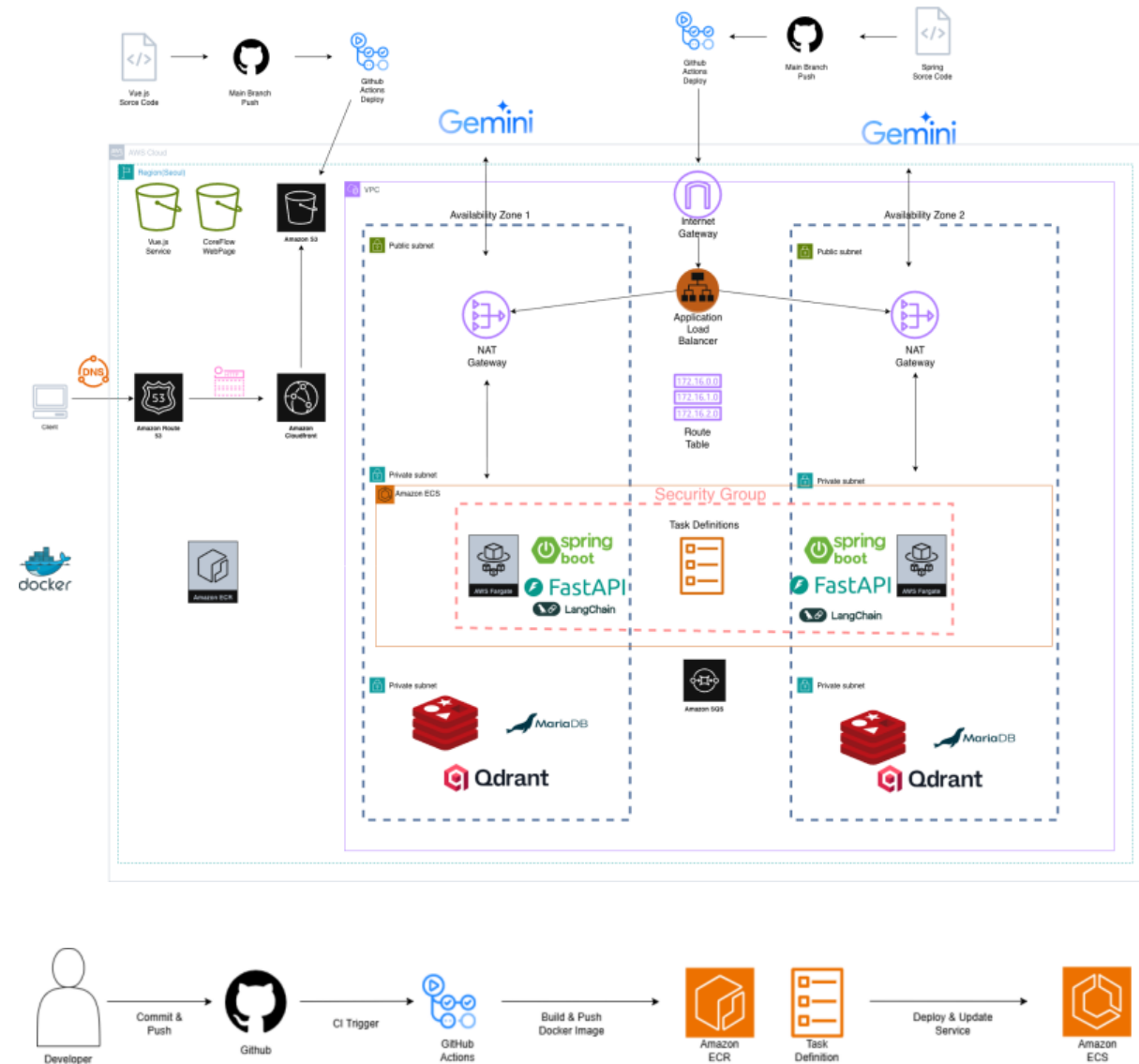
BACKEND TECHNOLOGY

리스크 분석 및 PDF 처리



INFRASTRUCTURE & DEPLOYMENT

인프라 및 배포 구조



GITHUB ACTIONS, ECS를 활용한 ROLLING UPDATE 방식의 CI/CD 파이프라인 구축
 AWS CLOUDWATCH를 활용한 장애 로그 모니터링 및 대응 마련
 PRIVATE SUBNET & NAT GATEWAY로 안전한 아웃바운드 통신 구성

TROUBLESHOOTING

트러블슈팅

CLOUDFRONT(S3) 프론트와 ECS(ALB) 백엔드 연동 후 로그인 실패 이슈

현상

- 로그인 화면은 정상 표시
- 로그인 시 403 / 401 / ERR_CONNECTION_REFUSED 반복 발생

환경

- FRONT : VUE + S3 + CLOUDFRONT
- BACK : SPRING BOOT + ECS(FARGATE) + ALB
- 인증 : JWT (STATELESS)

단일 원인이 아닌 “구성 불일치의 누적”

- 프론트 API URL이 빌드 타임에 LOCALHOST로 고정
- CLOUDFRONT에 /API/* → ALB 라우팅 없음
- ALB에 HTTPS 리스너 없음
- S3 OAC 버킷 정책 ARN 불일치
- SPA(HISTORY MODE) 라우팅 미처리

WHAT HAPPENED?
문제 상황

WHY?
원인

결과
RESULT

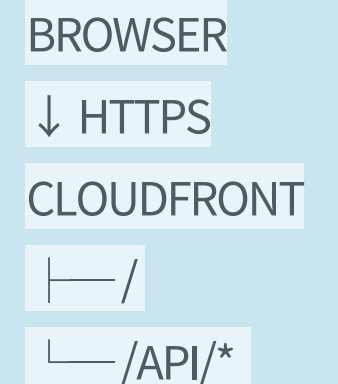
해결 구조
HOW?

결과

- 로그인 / 인증 API 정상 동작
- 403 / 401 / MIXED CONTENT 문제 완전 해소
- 프론트-백엔드 분리 배포 구조 안정화

CLOUDFRONT를 “단일 진입점”으로 재설계

- API 호출을 같은 오리진(/API)으로 통일
- CLOUDFRONT BEHAVIOR로 /API/* 프록시
- HTTPS 종료는 CLOUDFRONT에서만 담당
- 403/404 → INDEX.HTML (SPA 라우팅)
- 캐시 무효화로 설정 즉시 반영



TROUBLESHOOTING

트러블슈팅

SSE 알림 서비스 HIKARI 커넥션 점유/LEAK 발생

문제 상황

- SSE 구독을 오래 유지하거나 재연결을 반복한 뒤 간헐적으로:
 - HIKARI 커넥션 풀 고갈/대기 증가
 - LEAK DETECTION 트리거
(로그 STACK TRACE가 STREAM() 경로와 일치)
 - 클라이언트 504/응답 지연 발생 가능

트랜잭션 수명 관리 실패로 인한 이슈

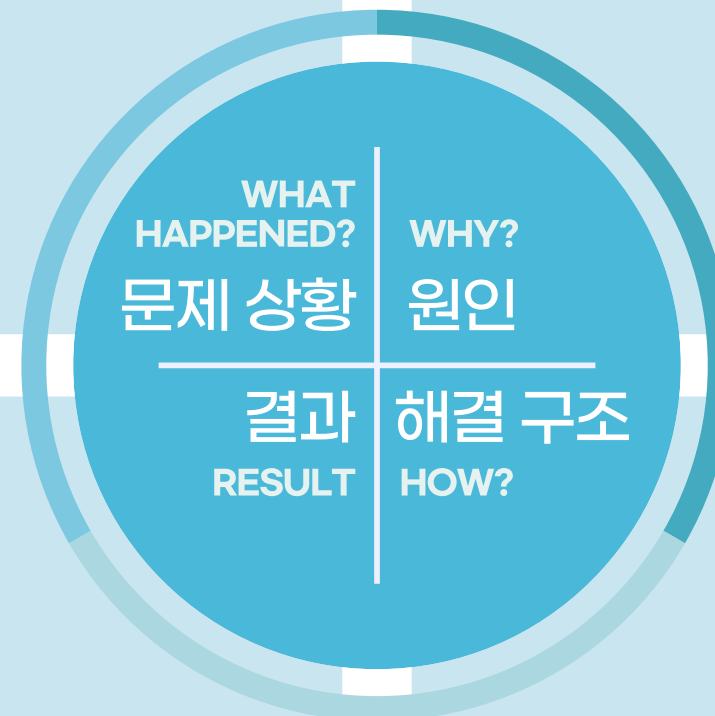
SSE 장기 연결과 DB 접근이 결합되며 트랜잭션 수명 장기화 위험 발생
재연결 폭주 시 동시 요청 급증으로 커넥션 풀 고갈 가능성 확대
이벤트 발행 기반 구조로 인해 전체 리소스 흐름이 복잡해짐

결과

- 알림 구독 서비스로 인한 서버 다운 문제 해결

해결 구조

- 알림 전달 방식을 SSE에서 POLLING으로 전환
- 기준 시점 이후 변경분만 조회하는 증분 조회 구조 도입
- 이벤트 전파 중심 구조 제거로 알림 처리 흐름 단순화
- 변경된 조회 방식에 맞춰 검증 기준 정비



TROUBLESHOOTING

트러블슈팅

낮은 RAG 검색 품질 개선

문제 상황

- 초기 시스템에서 검색 쿼리가 단일/고정 문장 위주, 근거 청크가 얇거나 설득력 낮음
- 다양한 리스크 시각 부족
 - 일정/품질/비용 등 관점별 쿼리 분리 없이
 - 주간보고/로그가 단일 관점으로만 검색됨

낮은 텍스트 추출 성능과 원시적인 파싱 기능

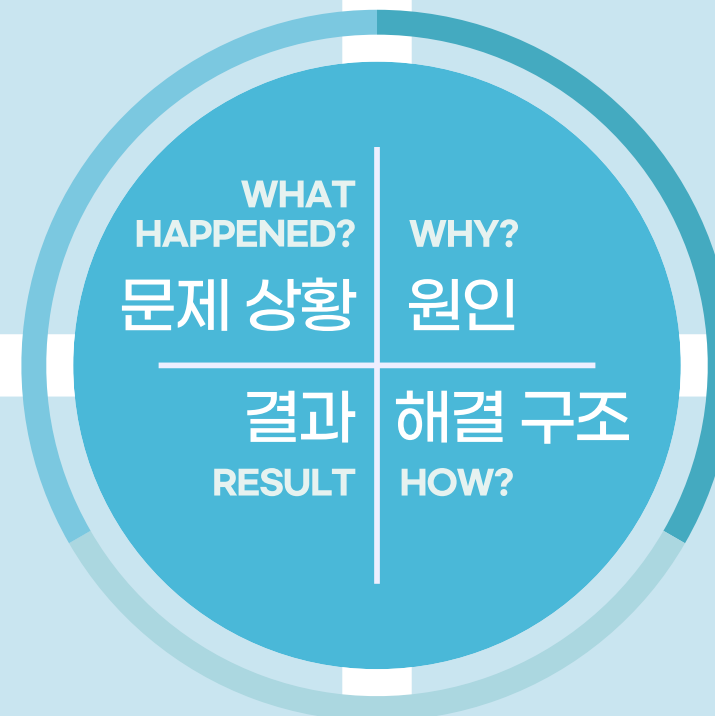
- 목차, 맷음글 등 리스크 분석에 필요 없는 내용들의 필터링 필요
- 낮은 텍스트 추출 성능과 원시적인 파싱 기능
- 쿼리 세밀도 부족
 - 리스크 유형/요소를 반영하지 않은 단일·일반 쿼리로 검색되어 관련성이 낮은 청크가 반환됨

결과

- 불필요 텍스트 제거로 벡터 근거 품질 개선
- 유형 기반 쿼리 분리로 검색 다양성·정합성 향상
- 리스크 리포트의 근거 설득력 및 추적성 강화

PDF 텍스트 추출 기능 강화 및 유형 기반 검색 도입

- PDF 텍스트 파싱 단계에서 불필요한 목차·헤더를 제거해 근거 품질을 개선
- PDF에서 리스크 유형/평가 요소를 추출하고, 이를 기반으로 유형별 쿼리 분리 검색



CHAPTER 4

PROJECT EXECUTION PROCESS & METHODOLOGY

프로젝트 수행 절차 및 방법

ALLOC 프로젝트의 수행 절차와 개발 방법을 소개합니다.

요구사항 분석부터 설계, 구현, 테스트까지의 전 과정을 체계적으로 관리하며,
기능 단위로 역할을 분리하고 단계적으로 개발을 진행하였습니다.

이를 통해 사용자 흐름을 중심으로 한 서비스 구현과 안정적인 시스템 구조를 완성했습니다.

PROJECT SCHEDULE

프로젝트 간트 차트

구분	기간	활동	비고
사전 기획	2025.12.16 ~ 12.25	프로젝트 주제 선정 및 기획 방향 수립	아이디어 도출
요구사항 정의	2025.12.26 ~ 01.02	요구사항 분석 및 기능 범위 정의	기획서, 요구사항 명세
도메인, 데이터, 시스템 설계	2025.01.03 ~ 01.11	ERD 설계, 시스템 아키텍처 및 WBS 수립	DDD 적용, DB 정규화, 역할 분담
UI/UX 설계	2026.01.12 ~ 01.17	화면 설계서 및 UI 흐름 정의	FIGMA 활용
API 설계	2026.01.18 ~ 01.19	API 명세서 작성	프론트/백엔드 협업
백엔드 개발	2026.01.19 ~ 01.30	핵심 비즈니스 로직 및 API 구현	CQRS 구조
프론트엔드 개발	2026.01.19 ~ 01.30	SPA 기반 화면 및 기능 구현	VUE.JS
통합 테스트	2026.01.31 ~ 02.04	프론트·백엔드 통합 테스트	안정성 확보 및 오류 수정
CI/CD 및 인프라	2026.02.02 ~ 02.06	CI/CD 파이프라인 설계	DOCKER, GITHUB ACTIONS

COLLABORATION & ROLE ASSIGNMENT

협업 방식 및 역할 분담



역할 분담 (설계)

담당 영역을 명확히 나눈 역할 기반 협업

- 프론트엔드 / 백엔드 / 인프라 역할 구분
- 기능 단위가 아닌 책임 영역 중심으로 분담
- 기획·설계 단계부터 각 파트가 함께 참여



커뮤니케이션 (개발)

실시간 소통과 기록 중심의 협업

- DISCORD를 통한 실시간 커뮤니케이션
- NOTION으로 기획, 정책, 결정사항 문서화
- 변경 사항은 팀 전체 공유 원칙 유지



협업 도구 활용 (관리)

협업 도구를 통한 작업 흐름 관리

- GITHUB ISSUES / PR 기반 작업 관리
- 기능 단위 PR로 변경 이력 추적
- 코드 리뷰를 통한 품질 관리
- ENTITY, API 공통 응답 등 공통 로직은 COMMON 브랜치에서 작업 후 머지하는 등 효율적인 GIT 전략 구축



NOW

피드백 & 개선 (운영)

피드백을 통한 지속적인 개선

- AWS CLOUD WATCH로 실시간 모니터링, 문제 발생시 디스코드로 공유
- 원인 공유 → 개선 반영 논의
- 협업 과정에서 나온 이슈를 다음 작업에 반영

CHAPTER 5
**PROJECT SUMMARY &
OUTCOME**

프로젝트 결과 및 회고

프로젝트를 수행하며 마주한 문제와 해결 과정을 되돌아보고,
팀과 개인의 성장을 중심으로 프로젝트를 정리합니다.

LIMITATIONS

아쉬운 점 / 한계

이번 프로젝트를 통해 기능 구현뿐 아니라 성능, 협업, 운영 관점에서의 개선 필요성을 명확히 인식할 수 있었습니다.



성능 개선 과정의
가시성 부족

#쿼리최적화

쿼리 리팩토링과 성능 개선이
진행되었으나 명확한 기준과
기록 없이 암묵적으로
수행되어 개선 의도가 코드
전반에 드러나지 못한 한계가
있었습니다.



협업을 위한
코드 리뷰 시간 부족

#코드리뷰

제한된 개발 일정으로 인해
충분한 코드 리뷰 시간을
확보하지 못해
협업 관점에서의 코드 품질
관리에 아쉬움이 남았습니다.



실시간 알림 구조의
리소스 부담

#SSE

SSE 기반 실시간 알림이 예상
보다 많은 리소스를 사용하며
운영 환경에서 장애 가능성을
고려해야 하는 한계를
확인했습니다.

FUTURE IMPROVEMENTS

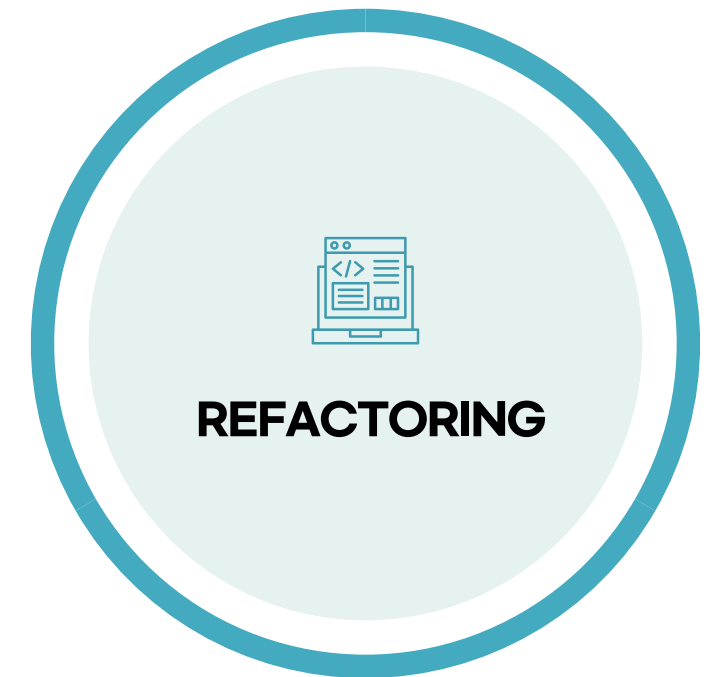
향후 개선 방향



**RISK TYPE
CLASSIFICATION**



**SSE
NOTIFICATION**



REFACTORING

AI, 알림, 코드 품질의 개선

RAG의 리스크 유형 분류 방식을 개선하고,
알림 서비스로 인한 부하를 해결,
마지막으로 리팩토링을 통해서
백엔드 서버의 성능을 최적화 필요

RAG 리스크 유형 분류 개선

RAG의 리스크 유형 분류 방식 개선
필요, 현재는 위험 관리 지침서 PDF
텍스트 추출 단계에서 RAG 유형 분류
를 위해 GEMINI에 질의 요청
유형 분류를 시스템 내재화할 수 있는
방법이 필요

SSE 알림 서비스 개선

로컬 환경에서 검증시 SSE 알림
서비스가 무리 없이 동작했으나,
ECS 서버에 업로드 된 이후로는
장애의 원인이 되는 것을 확인
SSE의 장점을 살리고 부하를 줄이는
방안 모색

리팩토링 및 성능 최적화

애자일한 프로젝트 진행으로 인해
백엔드 성능 측정 및 최적화 과정이
조금 미흡했다고 판단,
추후 K6를 활용하여 백엔드 성능을
측정하고 리팩토링을 진행할 예정

PROJECT RETROSPECTIVE

프로젝트 회고



김현수

PM, BACKEND, ML

로컬에서는 드러나지 않던 문제가 서비스 환경에서는 다수의 장애로 확산될 수 있음을 경험했고, 이를 계기로 사전에 장애 요인을 식별·차단할 수 있도록 소프트웨어 아키텍처를 설계하는 습관의 필요성을 명확히 인식하게 되었습니다.



김동리

BACKEND, ML

프로젝트 및 자원 배치 기능 설계 과정에서 API 설계 시 도메인 식별자 전략의 중요성을 인식했습니다. 또한 정량 평가의 한계를 인지하고, 학습 기반 추천으로의 확장 가능성을 검토했습니다. 아울러 검색 구조를 의미 기반으로 설계해, 향후 지능형 자동화의 확장 여지를 고려했습니다.



김명진

BACKEND, SECURITY

단순 기능 구현을 넘어서 서비스 분리 과정에서의 설정 충돌, 환경 변수 관리, 배포 오류, 테스트 데이터 문제 등 다양한 시행착오를 겪으며 문제 해결 능력을 키울 수 있었고, GIT 브랜치 전략과 협업 방식의 중요성을 체감할 수 있었습니다.



박연수

PM, FRONTEND

실무에서 근무하며 실제로 필요하다고 느꼈던 기능들을 직접 기획하고 구현할 수 있어 의미 있는 경험이었습니다. 사용자 흐름과 현업 상황을 고려한 기능 설계를 통해, 실무 관점에서 서비스를 바라보는 시각을 넓힐 수 있었습니다.



배창민

BACKEND, INFRA

ALLOC 최종 프로젝트에서 ECS·S3/CLOUDFRONT CI/CD로 배포 표준화와 운영 관점을 익혔습니다. 알림·캘린더 구현 중 SSE로 발생한 DB 커넥션 풀 고갈을 추적해 POLLING으로 전환하며 운영 제약과 관측성, 부하 테스트의 중요성을 깨달았습니다.

T E A M M O I R A I

ALLOC

THANK
YOU

함께해주셔서 감사합니다



팀원 : 김동리 · 김명진 · 김현수 · 박연수 · 배창민